

UNIVERSIDAD INTERNACIONAL DEL ECUADOR



TEMA

Aprendizaje Autónomo 1 Selección de Sistemas de Gestión empresarial

Asignatura

Programación Orientada a Objetos

AUTOR

Cristofer Jhonatan Tupiza Cadena

Quito-Ecuador

Julio – 2026

Introducción

La forma en que las empresas manejan sus procesos comerciales ha cambiado mucho gracias a la transformación digital. Esto ha llevado a que las empresas adopten herramientas tecnológicas para optimizar sus operaciones, mejorar la comunicación con los clientes y aumentar las oportunidades de negocio. El comercio electrónico se ha vuelto muy importante para vender productos y servicios porque permite que los consumidores accedan a ellos todo el tiempo y hacer transacciones desde cualquier lugar.

Este proyecto busca analizar y diseñar SmartCommerce, un sistema para gestionar tiendas virtuales de manera automática y controlada. Este sistema permitirá a las empresas gestionar sus ventas de manera centralizada mediante módulos especiales para clientes, productos, inventario, pedidos y reportes. Con esto, se busca mejorar la organización de la información, reducir errores y proporcionar herramientas para tomar decisiones informadas en un entorno comercial digital.

Desde el punto de vista tecnológico, se utilizará el lenguaje de programación Go para desarrollar el sistema. Esta herramienta es conocida por ser eficiente, simple y capaz de desarrollar aplicaciones escalables y de alto rendimiento. También se aplicarán principios de programación funcional para construir una arquitectura modular basada en funciones reutilizables y responsabilidades claras. Esto permitirá mejorar el mantenimiento del software, facilitar futuras ampliaciones y garantizar una estructura de código organizada y eficiente.

Este proyecto no solo es una solución tecnológica para la gestión de procesos comerciales, sino también una oportunidad para aplicar conocimientos de Ingeniería de Software, como análisis de requerimientos, diseño de sistemas y bases de datos.

Análisis del Sistema

El sistema SmartCommerce corresponde a una plataforma web orientada a la gestión de procesos de comercio electrónico. Su finalidad es permitir la administración centralizada de información relacionada con usuarios, productos, inventario, pedidos y ventas dentro de un mismo entorno tecnológico. La implementación de esta solución busca optimizar los procesos comerciales mediante la automatización de tareas que tradicionalmente requieren intervención manual, mejorando la eficiencia operativa y reduciendo la probabilidad de errores administrativos.

La plataforma estará dirigida tanto a clientes como a administradores. Los clientes podrán acceder al catálogo de productos, realizar compras y consultar el estado de sus pedidos. Los administradores tendrán acceso a herramientas de gestión para controlar inventarios, actualizar información de productos, monitorear ventas y generar reportes que permitan evaluar el rendimiento comercial de la organización. De esta manera, el sistema constituye una solución integral que facilita la administración de una tienda virtual moderna.

Requerimientos del Sistema

Los requerimientos del sistema son las acciones y servicios que el sistema debe proporcionar para satisfacer las necesidades de los usuarios.

Registro de usuarios

El sistema debe de permitir el registro de nuevos clientes mediante un formulario de inscripción.

Inicio de sesión

El sistema debe permitir la identificación de usuarios mediante un correo electrónico y contraseña.

Gestión de productos

El administrador deberá poder crear, modificar y eliminar productos registrados dentro de un catálogo.

Gestión de inventario

El sistema deberá permitir controlar la cantidad de productos disponibles para la venta.

Gestión de compras

Los clientes podrán agregar productos al carrito de compras y generar pedidos.

Gestión de pedidos

El sistema deberá almacenar el historial de compras realizadas por cada cliente.

Generación de reportes o problemas que se podrá visualizar estadísticas de los problemas relacionadas con ventas y productos.

Seguridad

Las contraseñas deberán almacenarse mediante mecanismos de cifrados seguros.

Rendimiento

El sistema deberá funcionar alas solicitudes de los usuarios en tiempos adecuados.

Disponibilidad

El sistema deberá estar disponible para acceso mediante navegador web.

Escalabilidad

La arquitectura deberá permitir futuras ampliaciones de funcionalidades.

Usabilidad

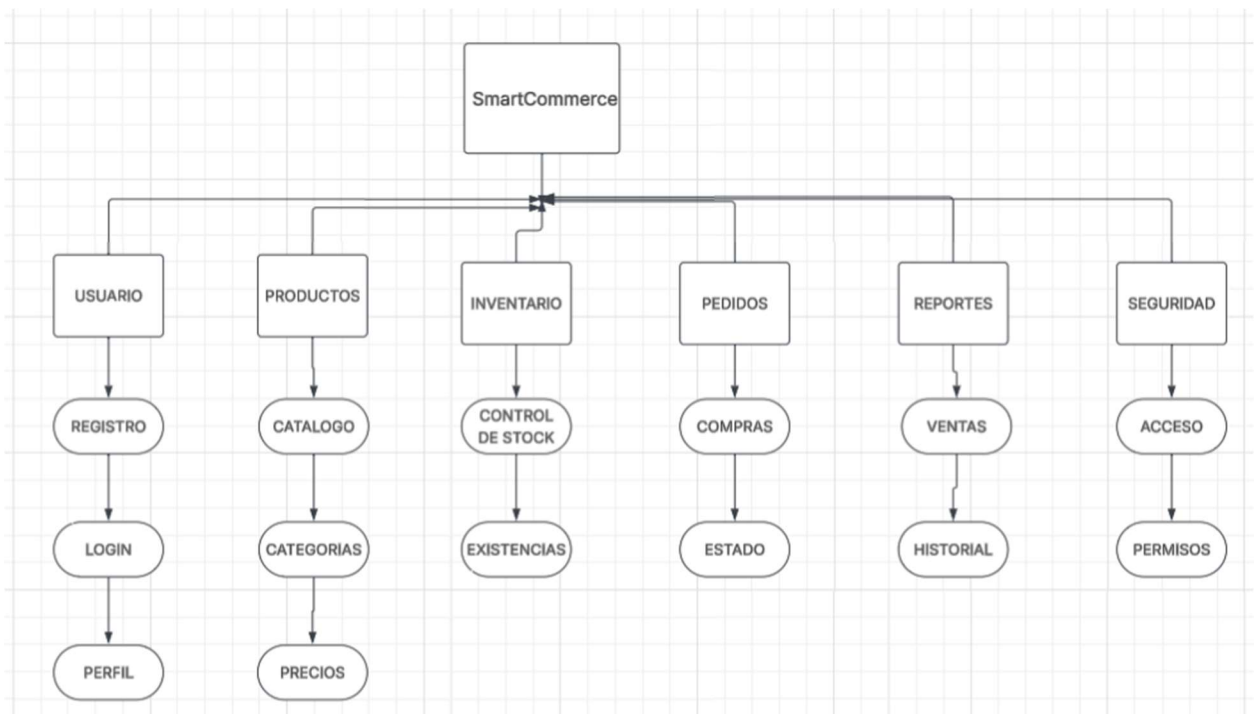
La interfaz deberá ser intuitiva y sencilla de utilizar.

Estructura del Sistema

La estructura de gestión de SmartCommerce se encuentra organizada en módulos independientes que permiten administrar de manera eficiente cada uno de los procesos involucrados en el comercio electrónico. Esta división favorece la escalabilidad, el mantenimiento y la reutilización de componentes durante el desarrollo del software.

Figura 1

Estructura General del Sistema SmartCommerce



La estructura funcional del sistema está compuesta por seis módulos principales. Cada módulo posee responsabilidades específicas dentro de la operación general del sistema. Esta separación permite que cada componente pueda evolucionar independientemente sin afectar el resto de la plataforma.

Funcionalidades de los módulos

Gestión de los usuarios

Estos módulos administran la información de los usuarios registrados y controlados en el acceso del sistema.

Funcionalidades:

Registro de usuarios.

Inicio de sesión.

Gestión de perfiles.

Recuperación de contraseñas.

Gestión de roles y permisos.

Gestión de productos

Permitir administrar el catalogo comercial disponible para los clientes.

Funcionalidades:

Registro de productos.

Edición de información.

Control de categorías.

Gestión de precios.

Búsqueda y filtrado.

Gestión de inventario

Controla la disponibilidad de productos dentro del sistema.

Funcionalidades:

Actualización de stock.

Registro de movimientos.

Control de existencias.

Alertas por bajo inventario.

Gestión de pedidos

Administra el ciclo de cada compra realizada por los clientes.

Funcionalidades:

Creación de pedidos.

Confirmación de compras.

Actualización de estados.

Historial de pedidos.

Gestión de reportes

Genera información de estrategias para la toma de decisiones.

Funcionalidades

Reportes de ventas.

Reportes de clientes.

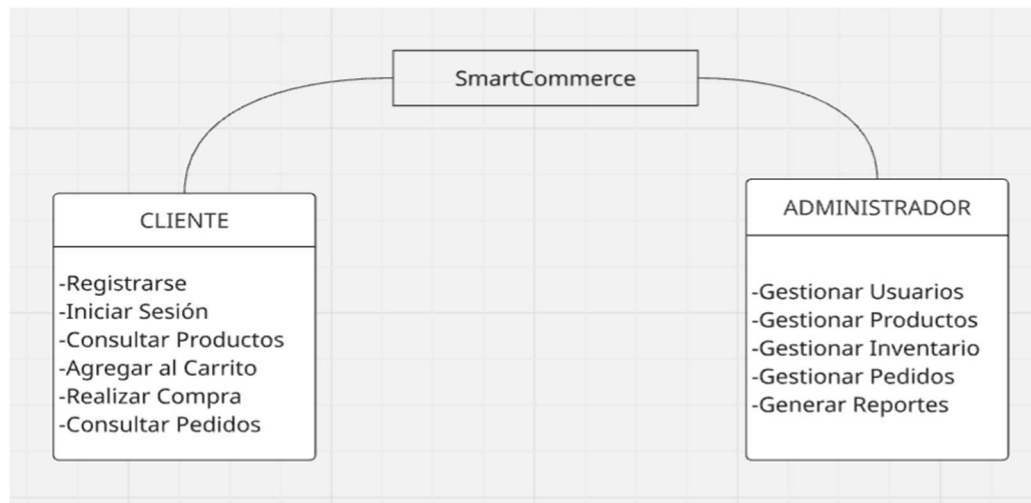
Reportes de productos.

Reportes de inventario.

Modelado del sistema

Figura 2

Casos de Uso del Sistema



El diagrama de casos de uso identifica las acciones que pueden realizar los actores principales dentro del sistema. Los clientes interactúan con las funcionalidades relacionadas con el proceso de compra, mientras que los administradores gestionan la operación interna de la plataforma.

Arquitectura general del Sistema

Figura 3

Arquitectura por Capas

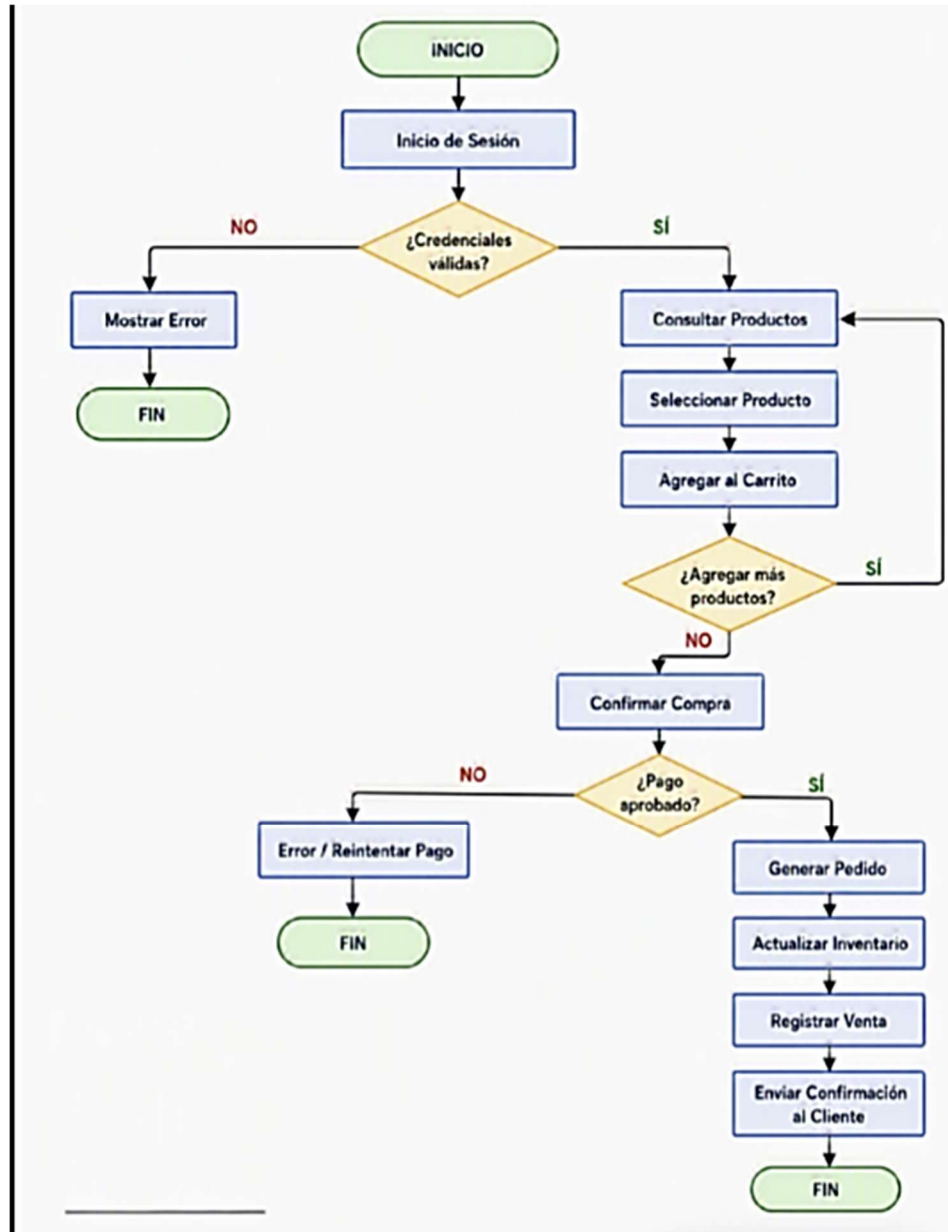


La arquitectura propuesta sigue un modelo de capas que separa la interfaz de usuario, la lógica de negocio y la persistencia de datos. Esta organización facilita el mantenimiento y la escalabilidad de la plataforma.

Flujo General del Proceso de Compra

Figura 4

Flujo del Negocio

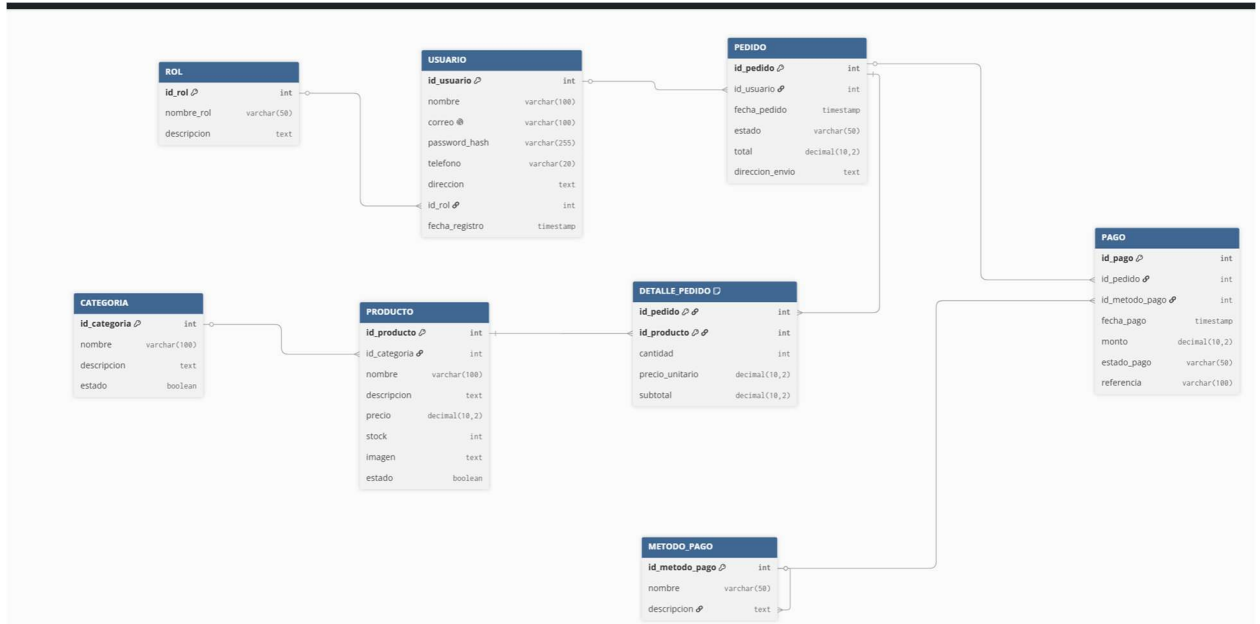


Este flujo representa el proceso que sigue un cliente desde el acceso al sistema hasta la generación de una compra exitosa.

Modelo Conceptual de Datos

Figura 5

Modelo Entidad Relación



El modelo entidad-relación identifica las entidades principales que compondrán la base de datos del sistema y las relaciones existentes entre ellas

Project SmartCommerce {

 database_type: "PostgreSQL"

}

Table USUARIO {

 id_usuario int [pk]

 nombre varchar(100)

 correo varchar(100) [unique]

password_hash varchar(255)

telefono varchar(20)

direccion text

id_rol int

fecha_registro timestamp

}

Table ROL {

id_rol int [pk]

nombre_rol varchar(50)

descripcion text

}

Table CATEGORIA {

id_categoria int [pk]

nombre varchar(100)

descripcion text

estado boolean

}

Table PRODUCTO {

```
id_producto int [pk]

id_categoria int

nombre varchar(100)

descripcion text

precio decimal(10,2)

stock int

imagen text

estado boolean

}
```

Table PEDIDO {

```
id_pedido int [pk]

id_usuario int

fecha_pedido timestamp

estado varchar(50)

total decimal(10,2)

direccion_envio text

}
```

Table DETALLE_PEDIDO {

id_pedido int

id_producto int

cantidad int

precio_unitario decimal(10,2)

subtotal decimal(10,2)

indexes {

(id_pedido, id_producto) [pk]

}

}

Table METODO_PAGO {

id_metodo_pago int [pk]

nombre varchar(50)

descripcion text

}

Table PAGO {

id_pago int [pk]

id_pedido int

```
id_metodo_pago int

fecha_pago timestamp

monto decimal(10,2)

estado_pago varchar(50)

referencia varchar(100)

}
```

RELACIONES

Ref: USUARIO.id_rol > ROL.id_rol

Ref: PEDIDO.id_usuario > USUARIO.id_usuario

Ref: PRODUCTO.id_categoria > CATEGORIA.id_categoria

Ref: DETALLE_PEDIDO.id_pedido > PEDIDO.id_pedido

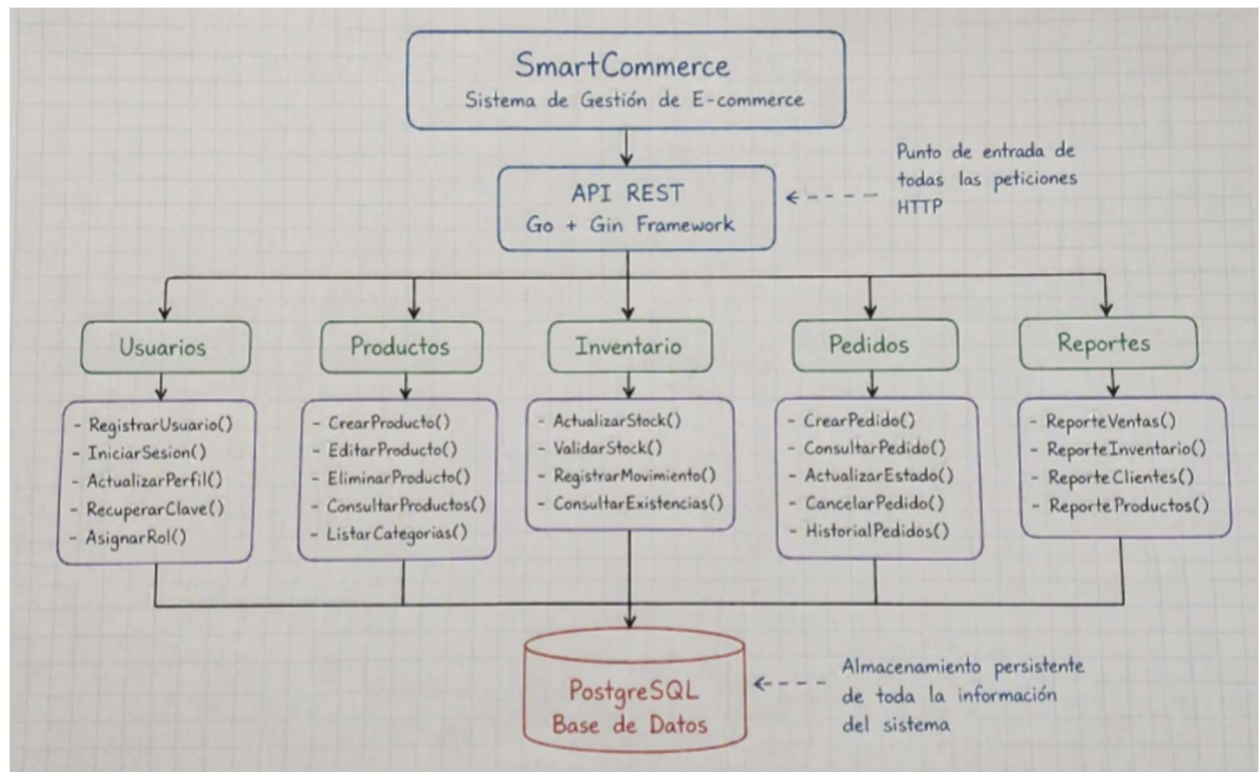
Ref: DETALLE_PEDIDO.id_producto > PRODUCTO.id_producto

Ref: PAGO.id_pedido > PEDIDO.id_pedido

Ref: PAGO.id_metodo_pago > METODO_PAGO.id_metodo_pago

Aplicación de Programación Funcional en Go

El sistema SmartCommerce será desarrollado utilizando Go como lenguaje principal. Se aplicarán principios de programación funcional mediante la utilización de funciones específicas y reutilizables para cada proceso de negocio.



Tecnologías y Herramientas del Proyectodam

La construcción del sistema SmartCommerce requiere la utilización de tecnologías modernas que permitan garantizar rendimiento, escalabilidad, seguridad y facilidad de mantenimiento. La selección de estas herramientas se realizó considerando las necesidades funcionales del sistema y las características propias del entorno de desarrollo definido para el proyecto.

El lenguaje de programación principal será **Go (Golang)** debido a su eficiencia en el procesamiento de solicitudes concurrentes, su simplicidad de sintaxis y su capacidad para desarrollar aplicaciones de alto rendimiento. Para la construcción de la API REST se empleará el framework **Gin**, el cual facilita la gestión de rutas, peticiones HTTP y middleware dentro de aplicaciones web modernas.

En cuanto a la persistencia de datos, se utilizará **PostgreSQL**, un sistema gestor de bases de datos relacional ampliamente utilizado en aplicaciones empresariales por su estabilidad, seguridad y capacidad para manejar grandes volúmenes de información. Para garantizar la seguridad del sistema se incorporarán mecanismos de autenticación mediante **JSON Web Tokens (JWT)** y cifrado de contraseñas utilizando la librería **bcrypt**.

Finalmente, el proyecto utilizará herramientas como Git y GitHub para el control de versiones, Visual Studio Code como entorno de desarrollo y Lucidchart para la elaboración de diagramas de análisis y diseño del sistema.

Configuración y Organización del Repositorio

La gestión del código se hará con un repositorio en GitHub. Esto permite que los miembros del equipo trabajen juntos, sigan los cambios y documenten el progreso del proyecto. La estructura del repositorio se diseñó para que sea modular y esté organizada, como se hace con aplicaciones hechas en Go.

Descripción de Directorios

README.md

Contendrá la descripción general del proyecto, instrucciones de instalación, tecnologías utilizadas y guía de uso para los desarrolladores.

go.mod

Archivo encargado de gestionar las dependencias y módulos utilizados por el lenguaje Go.

.gitignore

Permitirá excluir archivos temporales, configuraciones locales y elementos que no deben almacenarse en el repositorio.

cmd/

Contendrá el punto de entrada principal de la aplicación, específicamente el archivo `main.go`, encargado de iniciar el servidor y cargar las configuraciones necesarias.

internal/

Directorio principal donde estará implementada la lógica de negocio del sistema.

- **handlers/**: gestión de solicitudes HTTP.
- **services/**: reglas de negocio del sistema.
- **repository/**: acceso y manipulación de datos.
- **models/**: definición de estructuras y entidades.
- **middleware/**: autenticación, validación y seguridad.

database/

Almacenará configuraciones, conexiones y scripts relacionados con PostgreSQL.

docs/

Contendrá documentación técnica, diagramas y recursos asociados al proyecto.

tests/

Se utilizará para almacenar pruebas unitarias e integración del sistema.

La forma en que están organizadas las carpetas permite separar las responsabilidades de cada parte. Los módulos que tienen que ver con los usuarios, los productos, el inventario, los pedidos y los reportes estarán juntos en la carpeta principal de lógica de negocio. Las configuraciones, la documentación y las pruebas se mantendrán en lugares separados para que sea más fácil administrar el sistema.

Relación con la Disciplina

El desarrollo de SmartCommerce combina conocimientos de diferentes áreas de la Ingeniería de Software y las Tecnologías de la Información. Primero, se utilizan conceptos de análisis y diseño de sistemas para identificar las necesidades de los usuarios de la plataforma. Esto incluye los requerimientos funcionales y no funcionales. Además, se crea una estructura modular que facilita la organización de las funcionalidades y mejora la mantenibilidad del software.

En cuanto a la programación, el proyecto utiliza el lenguaje Go para implementar la lógica de negocio. Se aplican funciones, paquetes, estructuras condicionales y ciclos iterativos para construir una solución organizada y eficiente. Esto se alinea con los principios de programación funcional.

El sistema también incorpora conocimientos de bases de datos. Se diseña un modelo entidad-relación y se utiliza PostgreSQL para almacenar la información. Esto permite gestionar

los datos de usuarios, productos, inventarios y pedidos de manera adecuada. La información es accesible y segura.

La arquitectura de software es otro aspecto importante. La solución sigue una estructura por capas que separa la presentación, la lógica de negocio y la persistencia de datos. Esto facilita la escalabilidad, reutilización de componentes y mantenimiento del sistema.

Finalmente, SmartCommerce integra conceptos del comercio electrónico. Permite la gestión de productos, clientes, compras e inventarios dentro de una plataforma digital. Esto muestra la capacidad de aplicar conocimientos de diferentes disciplinas para resolver una problemática real con una solución tecnológica moderna.

Conclusión

El desarrollo de la etapa de planificación de SmartCommerce nos permitió ver lo importante que es hacer un análisis antes de empezar a construir una solución tecnológica. En este proyecto, identificamos las necesidades principales de una plataforma de comercio electrónico. Definimos su estructura y los módulos que necesita para funcionar. También decidimos qué tecnologías vamos a usar.

Nuestra propuesta incluye procesos como la gestión de usuarios, productos, inventarios, pedidos y reportes. Establecimos una organización clara para que sea fácil administrar la información y que los clientes y administradores puedan interactuar. Además, elegimos Go como lenguaje de programación principal. Esto nos permitirá desarrollar una aplicación que sea eficiente, escalable y que pueda mejorar en el futuro.

En esta fase, aplicamos conocimientos de análisis de requerimientos, modelado de sistemas, bases de datos, arquitectura de software y programación funcional. Esto nos mostró que podemos relacionar conceptos teóricos con problemas reales en el ámbito empresarial. También creamos diagramas, estructuras modulares y modelos de datos. Esto nos dio una visión completa del proyecto y nos ayudó a tomar decisiones técnicas para las siguientes etapas.

En resumen, SmartCommerce es una propuesta sólida para gestionar procesos de comercio electrónico. Es un ejemplo práctico de cómo se pueden aplicar los conocimientos adquiridos en la asignatura. Esto fortalece competencias técnicas y analíticas fundamentales en la formación en Ingeniería de Software.

Bibliografía

Biu.us. (s.f.). Obtenido de Maestría en Ciencias en Ingeniería de Software Informático:
<https://masters.biu.us/maestria-de-ciencia-en-ingenieria-de-software-informatico/>

Gin Web Framework. (s.f.). Obtenido de Gin Web Framework: <https://gin-gonic.com/es/>

GitHub. (2025). *GitHub Documentation*. Obtenido de <https://docs.github.com/>

Losada, N. (05 de 25 de 2022). *PostgreSQL y PostGIS: Qué son y cómo se relacionan*. Obtenido de Geoinnova: <https://geoinnova.org/blog-territorio/postgresql-y-postgis-que-son-y-como-se-relacionan/>

SerpApi: Golang integration. (s.f.). Obtenido de SerpApi: <https://serpapi.com/integrations/go>